

حتماً. برای اینکه واقعاً این ۸ الگوریتم را بفهمی نه اینکه فقط تعریفشان را حفظ کنی، اول دو خانواده را از هم جدا می‌کنیم:

Clustering

یعنی داده label ندارد و می‌خواهیم بفهمیم آیا sample‌ها به گروه‌های طبیعی تقسیم می‌شوند یا نه.

چهار الگوریتم اصلی که بررسی می‌کنیم:

K-Means, DBSCAN, Gaussian Mixture Model (GMM), Hierarchical Clustering

و:

Dimensionality Reduction

یعنی داده ممکن است صدها یا هزاران feature داشته باشد و می‌خواهیم آن را با تعداد کمتری dimension نمایش دهیم، بدون اینکه اطلاعات مهم آن بیش از حد از دست برود.

چهار روش:

PCA, UMAP, t-SNE, LDA

یک اصلاح مهم: LDA یا Linear Discriminant Analysis برخلاف PCA/UMAP/t-SNE یک روش supervised است و برای اجرا به label نیاز دارد.

بخش صفر: اصلاً Clustering یعنی چه؟

فرض کن برای ۱۰۰۰ بیمار این feature‌ها را داری:

$X = [\text{Age, BMI, Blood Pressure, Glucose, Cholesterol, ...}]$

ولی هیچ label‌ای نداری که بگویند:

Patient 1 → Group A
Patient 2 → Group B
...

می‌خواهی ببینی آیا خود داده به طور طبیعی به چند phenotype تقسیم می‌شود.

مثلاً بعد از clustering شاید بفهمی:

Cluster 1 → young, low BMI, low glucose
Cluster 2 → older, high glucose
Cluster 3 → obese, high blood pressure

یعنی الگوریتم بدون داشتن class label, structure داخل داده را پیدا کرده است.

1. K-Means

K-Means چیست؟

K-Means یکی از ساده‌ترین و مهم‌ترین clustering algorithm ها است.

ایده اصلی:

داده را به K گروه تقسیم کن، به طوری که اعضای هر گروه تا حد امکان به مرکز cluster خود نزدیک باشند.

آن مرکز را:

Centroid

می‌نامیم.

مثال خیلی ساده

فرض کن مختصات ۶ نقطه را داریم:

$$A = (1, 1)$$

$$B = (1, 2)$$

$$C = (2, 1)$$

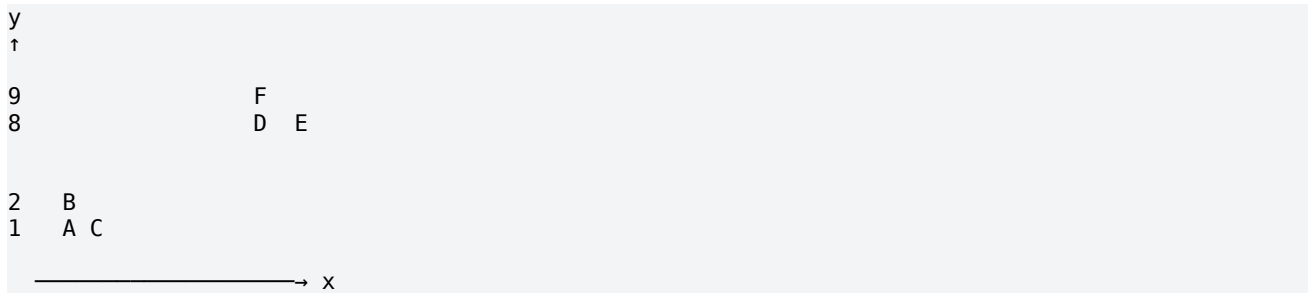
و:

$$D = (8, 8)$$

$$E = (9, 8)$$

$$F = (8, 9)$$

از روی چشم مشخص است:



احتمالاً دو cluster داریم.

پس:

$$K = 2$$

K-Means دقیقاً چگونه کار می‌کند؟

فرض کنیم ابتدا دو centroid تصادفی انتخاب کنیم:

$$\mu_1$$

و:

$$\mu_2$$

مرحله Assignment

برای هر sample فاصله تا centroidها را حساب می‌کنیم.

مثلاً برای:

$$A = (1, 1)$$

اگر:

$$d(A, \mu_1) = 0.5$$

و:

$$d(A, \mu_2) = 10$$

باشد، A وارد Cluster 1 می شود.

مرحله Update

بعد centroid جدید cluster را حساب می کنیم:

$$\mu_k = \frac{1}{N_k} \sum_{x_i \in C_k} x_i$$

یعنی average تمام نقاط داخل cluster.

مثلاً برای:

$$A = (1, 1), B = (1, 2), C = (2, 1)$$

centroid می شود:

$$\begin{aligned} \mu_1 &= \left(\frac{1 + 1 + 2}{3}, \frac{1 + 2 + 1}{3} \right) \\ &= (1.33, 1.33) \end{aligned}$$

سپس دوباره:

```
Assignment
↓
Update centroid
↓
Assignment
↓
Update centroid
↓
...
```

تا centroidها تقریباً دیگر جابه جا نشوند.

تابع هدف K-Means

K-Means تلاش می‌کند این مقدار را کمینه کند:

$$J = \sum_{k=1}^K \sum_{x_i \in C_k} ||x_i - \mu_k||^2$$

یعنی:

مجموع مربع فاصله تمام نقاط تا centroid cluster خودشان را کمینه کن.

این مقدار را معمولاً می‌گوییم:

Within-Cluster Sum of Squares

یا در sklearn:

Inertia

یعنی cluster خوب از دید K-Means چیست؟

مثلاً:

•••
•••

▲▲▲
▲▲▲

خوب است چون نقاط داخل هر cluster به هم نزدیک‌اند.

ولی:

• • •
•
▲ ▲ ▲

خوب نیست.

مهم‌ترین K: Hyperparameter

باید از قبل مشخص کنی:

$$K = ?$$

مثلاً:

```
KMeans(n_clusters=3)
```

این یکی از محدودیت‌های اصلی K-Means است.

چگونه K مناسب را انتخاب کنیم؟

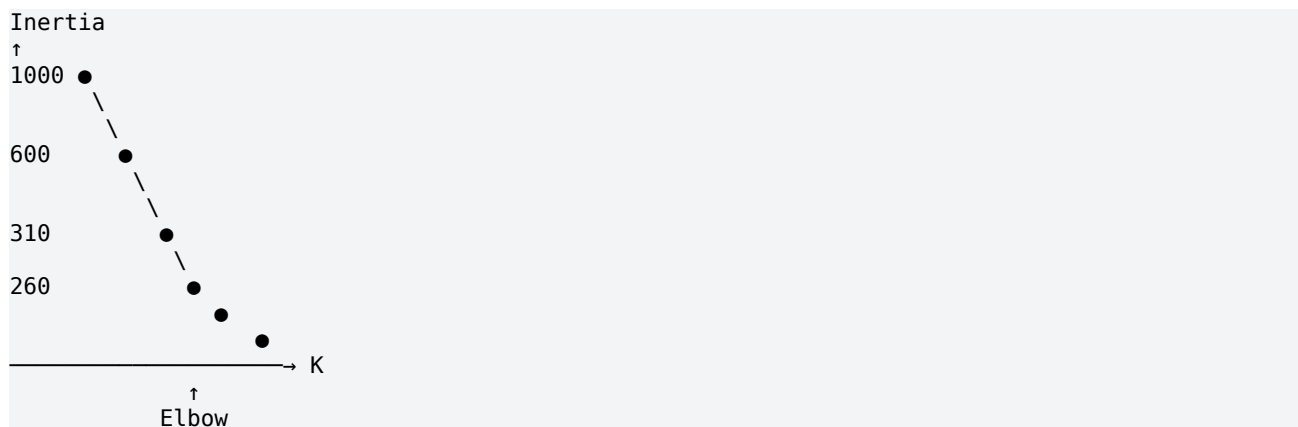
Elbow Method

برای Kهای مختلف Inertia را حساب می‌کنیم.

مثلاً:

K	Inertia
1	1000
2	600
3	310
4	260
5	230
6	210

نمودار تقریباً:



ممکن است:

$$K = 3$$

انتخاب منطقی باشد.

Silhouette Score

برای هر sample دو مقدار داریم.

a:
average distance آن sample با اعضای cluster خودش.

b:
average distance آن sample با نزدیکترین cluster دیگر.

فرمول:

$$s = \frac{b - a}{\max(a, b)}$$

مقدار:

$$-1 \leq s \leq 1$$

اگر:

$$s \approx 1$$

cluster خوب.

اگر:

$$s \approx 0$$

sample روی مرز clusterهاست.

اگر:

$$s < 0$$

احتمالاً sample به cluster اشتباه assign شده.

++K-Means چیست؟

یک مشکل K-Means این است که initialization تصادفی centroidها می تواند نتیجه را خراب کند.

فرض کن هر دو centroid اولیه در یک گوشه قرار بگیرند.

الگوریتم ممکن است وارد local optimum ضعیفی شود.

centroid ++K-Meansهای اولیه را هوشمندانه تر انتخاب می کند تا از هم فاصله داشته باشند.

در sklearn معمولاً initialization مناسب به صورت پیش فرض استفاده می شود.

K-Means در Scaling

بسیار مهم.

فرض کن:

Age	= 20–80
Income	= 10,000–1,000,000
Score	= 0–1

K-Means عمدتاً Income را خواهد دید.

چون Euclidean distance تحت سلطه آن است.

پس معمولاً:

```
StandardScaler()
```

قبل از K-Means.

K-Means چه نوع clusterهایی را دوست دارد؟

تقریباً:

Spherical / compact clusters

مثلاً:

ooo
ooo
ooo

▲▲▲
▲▲▲
▲▲▲

کجا K-Means بد است؟

مثلاً Two Moons:

•••••
•••

▲
▲
▲▲▲▲

ممکن است K-Means اشتباه تقسیم کند چون دنبال centroid است.

همچنین برای clusterهای بسیار متفاوت از نظر size یا density مناسب نیست.

مثال واقعی

فرض کن اطلاعات غذایی داری:

Calories
Protein
Fat
Sugar
Fiber

بعد از standardization, K-Means با:

$$K = 3$$

ممکن است پیدا کند:

Cluster 1:
high protein
low sugar

Cluster 2:
high sugar
high fat

Cluster 3:
low calorie
high fiber

K-Means چه زمانی انتخاب خوبی است؟

وقتی تقریباً می‌دانی چند cluster داری، clusterها نسبتاً compact هستند، dataset بزرگ است و الگوریتم سریع می‌خواهی.

2. DBSCAN

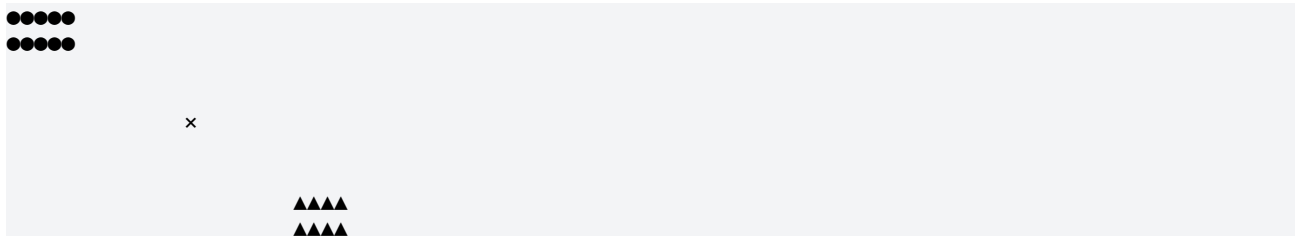
ایده کاملاً متفاوت

DBSCAN اصلاً centroid ندارد.

ایده:

Cluster یعنی ناحیه‌ای که density نقاط زیاد است.

پس:



DBSCAN می‌گوید:

● = Cluster 1
▲ = Cluster 2
× = Noise

DBSCAN چه مزیت بزرگی دارد؟

لازم نیست تعداد clusterها را بگوییم.

برخلاف:

$$K - Means(K)$$

در DBSCAN چیزی به اسم K نداریم.

دو Hyperparameter اصلی

eps

یا:

ϵ

شعاع neighborhood.

اگر فاصله دو point کمتر از eps باشد، neighbor محسوب می‌شوند.

min_samples

حداقل تعداد samples در eps-neighborhood تا region را dense بدانیم.

مثلاً:

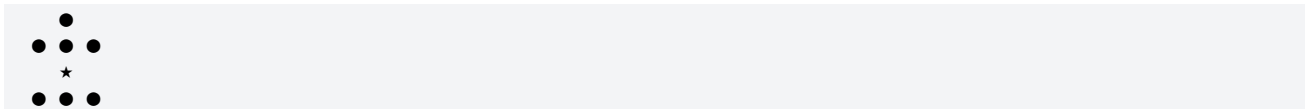
```
DBSCAN(  
    eps=0.5,  
    min_samples=5  
)
```

سه نوع Point

Core Point

اگر داخل شعاع eps حداقل min_samples باشد.

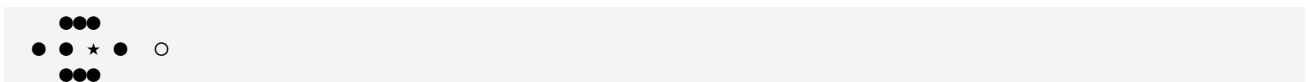
مثلاً:



★ core point است.

Border Point

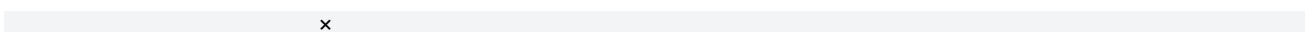
خودش neighbor کافی ندارد ولی به core point وصل است.



○ border point است.

Noise Point

نه core است و نه border.



در sklearn:

$label = -1$

DBSCAN چگونه cluster می‌سازد؟

یک core point پیدا می‌کند.

neighborهایش را به cluster اضافه می‌کند.

اگر یکی از neighborها خودش core باشد، neighborهای آن هم اضافه می‌شوند.

یعنی cluster رشد می‌کند:

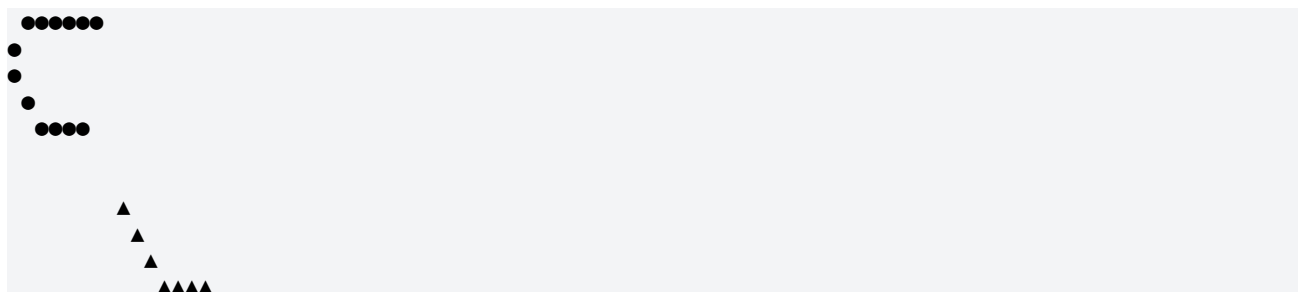
A → B → C → D → E

حتی اگر A و E مستقیم نزدیک هم نباشند.

چرا arbitrary-shaped cluster پیدا می‌کند؟

چون centroid محور تصمیم نیست.

مثلاً:



DBSCAN می‌تواند دو cluster را تشخیص دهد.

K-Means احتمالاً مشکل دارد.

eps خیلی کوچک

مثلاً:

$$eps = 0.1$$

ممکن است:

```
Noise Noise Noise
cluster1
cluster2
cluster3
cluster4
```

یعنی fragmentation زیاد.

eps خیلی بزرگ

مثلاً:

$$eps = 10$$

ممکن است:

```
Everything → one cluster
```

min_samples کم

الگوریتم permissive می شود.

مثلاً:

$$min_samples = 2$$

نقاط ضعیف هم ممکن است cluster تشکیل دهند.

min_samples زیاد

algorithm سختگیر می شود.

مثلاً:

$$min_samples = 30$$

region باید واقعاً dense باشد.

Noise بیشتر می شود.

k-distance plot

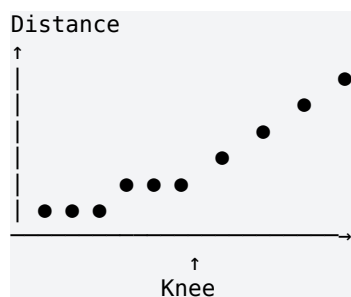
برای انتخاب eps بسیار مهم است.

مثلاً:

$$\text{min_samples} = 5$$

برای هر sample فاصله تا پنجمین nearest neighbor را حساب می کنیم.

بعد sort می کنیم.



اطراف knee candidate خوبی برای eps است.

مشکل اصلی DBSCAN

فرض کن cluster اول:



خیلی dense است.

Cluster دوم:



خیلی sparse.

یک eps ممکن است برای اول خوب باشد ولی برای دوم بد.

به این مشکل می‌گوییم:

Varying Density

اینجا HDBSCAN اغلب گزینه بهتری است.

High-dimensional data

DBSCAN در 500 یا 1000 dimension معمولاً مشکل پیدا می‌کند.

چون مفهوم distance ضعیف می‌شود.

این همان:

Curse of Dimensionality

است.

پس ممکن است:

```
768D embedding
↓
PCA → 30D
↓
DBSCAN
```

بهتر باشد.

ارزیابی DBSCAN

Silhouette Score قابل استفاده است، ولی معمولاً noise یعنی 1-label را جداگانه بررسی می‌کنیم.

همچنین می‌توان:

DBCV

را استفاده کرد که برای density-based clustering مناسب‌تر است.

باید درصد noise را هم بررسی کنی.

مثلاً:

```
2 clusters
Silhouette = 0.80
Noise = 85%
```


لزوماً عالی نیست؛ ممکن است algorithm تقریباً همه داده را دور انداخته باشد.

مثال واقعی DBSCAN

فرض کن GPS delivery drivers داری.

بیشتر locations در چند نقطه dense هستند:

Warehouse
Restaurant zone
Customer zone

و برخی points عجیب هستند.

DBSCAN می‌تواند هم location cluster و هم outlier را پیدا کند.

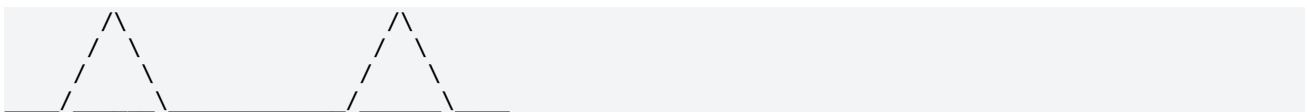
3. Gaussian Mixture Model — GMM

GMM چیست؟

GMM یک clustering algorithm احتمالاتی است.

فرض می‌کند data از ترکیب چند Gaussian distribution آمده است.

مثلاً:



دو Gaussian داریم.

فرمول

$$p(x) = \sum_{k=1}^K \pi_k \mathcal{N}(x|\mu_k, \Sigma_k)$$

هر component سه چیز اصلی دارد:

$$\mu_k$$

,mean

$$\Sigma_k$$

,covariance

:

$$\pi_k$$

mixing weight.

تفاوت بزرگ GMM و K-Means

K-Means:

Patient 15 → Cluster 2

تمام.

GMM:

Patient 15:

Cluster 1 = 0.10
Cluster 2 = 0.72
Cluster 3 = 0.18

پس GMM:

Soft Clustering

انجام می دهد.

Responsibility

احتمال تعلق sample i به component k:

$$\gamma_{ik} = P(z_i = k | x_i)$$

مثلاً:

[0.05, 0.90, 0.05]

یعنی membership خیلی واضح است.

ولی:

[0.33, 0.34, 0.33]

یعنی بسیار ambiguous.

GMM چطور آموزش می‌بیند؟

با الگوریتم:

Expectation Maximization — EM

E-Step

با Gaussian های فعلی membership probabilities را محاسبه می‌کنیم.

مثلاً:

x1 → [0.9, 0.1]
x2 → [0.7, 0.3]
x3 → [0.2, 0.8]

M-Step

با استفاده از این probabilities:

- Mean
- Covariance
- Mixing weights

را update می‌کنیم.

بعد دوباره:

E
↓
M
↓
E
↓
M
...

تا convergence.

Covariance Types

در sklearn چهار حالت مهم داریم.

Type	معنی
spherical	هر cluster شبیه کره
diag	variance جداگانه در هر dimension
tied	همه clusters یک covariance مشترک
full	هر cluster covariance کامل خودش

full انعطاف‌پذیرترین است ولی parameter بیشتری دارد.

GMM چه clusterهایی را دوست دارد؟

Elliptical Gaussian-like structures.

مثلاً:

```
****
*****
*****
*****
```

یا:

```
*
**
***
***
***
```

تعداد cluster را چطور تعیین کنیم؟

GMM مثل K-Means معمولاً نیاز دارد:

K

را مشخص کنیم.

اما اینجا دو metric بسیار مهم داریم:

AIC

$$AIC = -2 \log L + 2p$$

BIC

$$BIC = -2 \log L + p \log n$$

هر دو balance می کنند:

Good fit
versus
Too much model complexity

مقدار کمتر بهتر است.

مثلاً:

	K	BIC
1		8500

K	BIC
2	7100
3	6300
4	6000
5	6120
6	6300

پس:

$$K = 4$$

انتخاب خوبی است.

Overlap و GMM

اینجا GMM فوق العاده است.

مثلاً:

```
Cluster A
*****
*****
*****

*****
*****
Cluster B
```

در ناحیه overlap، به زور assignment نمی دهد.

می گوید مثلاً:

```
A = 0.55
B = 0.45
```

محدودیت GMM

اگر data مثل moon باشد:

```
•••••
•
•
•
•
•••••
```

Gaussian assumption ضعیف است.

GMM ممکن است مجبور شود چند Gaussian برای یک structure استفاده کند.

ارزیابی GMM

برای model selection:

AIC و BIC

برای model fit:

Log-Likelihood

برای uncertainty:

Posterior probability / entropy

و اگر hard label تولید کردی، Silhouette هم می‌تواند استفاده شود.

4. Hierarchical Clustering

ایده اصلی

Hierarchical clustering فقط یک partition نمی‌دهد.

بلکه یک:

Hierarchy of clusters

می‌سازد.

مثلاً:

```
All samples
├── Group A
│   ├── A1
│   └── A2
└── Group B
    ├── B1
    └── B2
```

دو نوع اصلی

Agglomerative

از پایین به بالا.

ابتدا هر sample یک cluster است.

بعد نزدیکترین cluster ها merge می شوند.

این رایج ترین حالت است.

Divisive

از بالا به پایین.

ابتدا همه data یک cluster هستند، سپس cluster ها تقسیم می شوند.

مثال Agglomerative

فرض کن چهار sample:

A B C D

ابتدا:

{A} {B} {C} {D}

A و B نزدیک اند:

{AB} {C} {D}

بعد C و D:

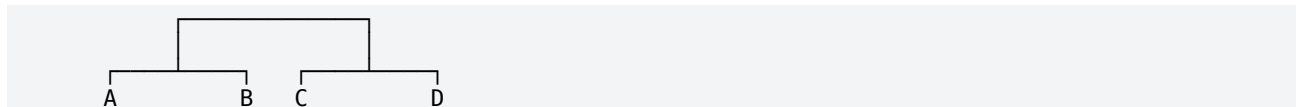
{AB} {CD}

بعد:

{ABCD}

Dendrogram چیست؟

نمایش درختی clustering.



ارتفاع merge نشان دهنده distance است.

Linkage یعنی چه؟

این یکی از مهم ترین مفاهیم Hierarchical Clustering است.

وقتی دو cluster داریم، باید تعریف کنیم distance میان دو cluster چیست.

Single Linkage

کمترین فاصله بین دو عضو:

$$d(A, B) = \min d(a, b)$$

مزیت:

shape پیچیده را می تواند دنبال کند.

مشکل:

Chaining effect

ممکن است clusterها با یک زنجیره باریک به هم وصل شوند.

Complete Linkage

بیشترین فاصله:

$$d(A, B) = \max d(a, b)$$

clusterهای compact تری می دهد.

Average Linkage

average تمام pairwise distances.

متعادل تر است.

Ward Linkage

merge ای را انتخاب می کند که کمترین افزایش در within-cluster variance ایجاد کند.

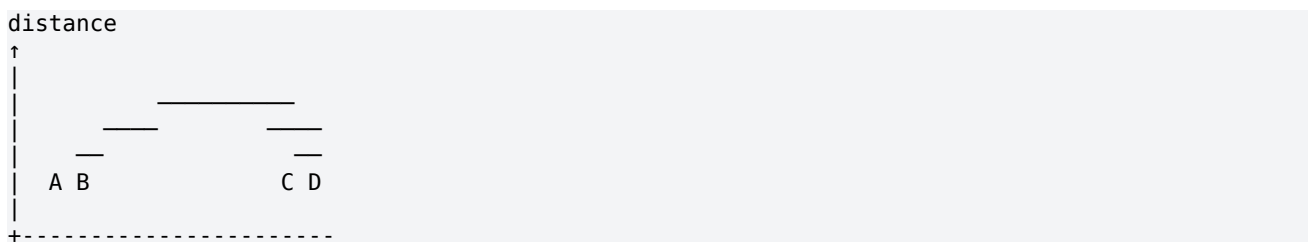
خیلی رایج است.

Ward معمولاً با Euclidean distance استفاده می شود.

تعداد cluster را چطور انتخاب کنیم؟

از dendrogram cut می گیریم.

مثلاً:



اگر خط cut را در یک ارتفاع خاص بگذاری، مثلاً:

A+B
C+D

دو cluster خواهی داشت.

مزیت مهم Hierarchical Clustering

لازم نیست از ابتدا K را تعیین کنی.

اول hierarchy می سازی و بعد تصمیم می گیری کجا cut کنی.

مشکل

برای dataset های بسیار بزرگ، pairwise distance computation می تواند گران باشد.

مثلاً برای:

$$n = 1,000,000$$

Hierarchical clustering معمولاً انتخاب خوبی نیست.

مثال واقعی

Gene expression data.

مثلاً صد ژن و صد بیمار.

Hierarchical clustering بسیار رایج است چون می توانیم:

patients hierarchy
genes hierarchy

را ببینیم و heatmap بسازیم.

حالا Dimensionality Reduction

فرض کن data:

$$X \in R^{10000 \times 1000}$$

یعنی:

10,000 samples
1000 features

ممکن است:

- features redundant باشند
- noise داشته باشیم
- visualization غیرممکن باشد
- clustering سخت شود
- computation سنگین شود

می‌خواهیم:

$$1000D \rightarrow 50D$$

یا:

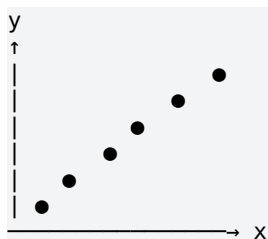
$$1000D \rightarrow 2D$$

5. PCA

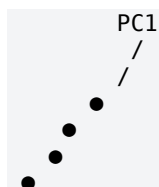
ایده اصلی

PCA دنبال directionهایی است که بیشترین variance داده در آنها قرار دارد.

فرض کن دو feature correlated:



PCA محور جدید:



را پیدا می کند.

PC1 بیشترین variance را دارد.

PC2 عمود بر PC1 است و بیشترین variance باقی مانده را توضیح می دهد.

ریاضیات PCA

ابتدا data را center می کنیم:

$$X_c = X - \mu$$

سپس covariance:

$$C = \frac{1}{n-1} X_c^T X_c$$

بعد:

$$Cv = \lambda v$$

Eigenvector:

$$v$$

جهت principal component.

Eigenvalue:

$$\lambda$$

مقدار variance آن direction.

Explained Variance

مثلاً:

PC1 = 50%
PC2 = 25%
PC3 = 12%
PC4 = 8%
PC5 = 5%

سه component اول:

$$50 + 25 + 12 = 87\%$$

variance کل را نگه داشته اند.

Cumulative Explained Variance

اگر threshold:

95%

باشد، به اندازه ای PC نگه می داریم تا:

$$\sum_{j=1}^k EV R_j \geq 0.95$$

PCA Feature Selection نیست

این خیلی مهم است.

PC1 ممکن است:

$$PC_1 = 0.6x_1 + 0.4x_2 - 0.3x_3 + \dots$$

باشد.

پس یک feature جدید است.

Loadings

ضرایب featureها در هر PC.

مثلاً:

$$PC_1 = 0.80Weight + 0.75BMI + 0.10Age$$

می توان نتیجه گرفت PC1 عمدتاً body-size-related است.

PCA linear است

اگر structure داده nonlinear باشد، PCA ممکن است نتواند آن را باز کند.

مثلاً Swiss Roll.

Hyperparameter اصلی

n_components

مثلاً:

`PCA(n_components=20)`

یا:

`PCA(n_components=0.95)`

معیار PCA

اصلی ترین:

Explained Variance Ratio

و:

Reconstruction Error

اگر:

$$X \rightarrow Z \rightarrow \hat{X}$$

آنگاه:

$$MSE(X, \hat{X})$$

هرچه کمتر، information loss کمتر.

Scaling

PCA به variance حساس است.

پس اگر unitها مختلف اند:

StandardScaler

↓

PCA

مثال

2048-dimensional CNN embedding:

2048D

↓

PCA

↓

50D

مثلاً:

96%

variance حفظ می شود.

6. UMAP

UMAP چیست؟

Uniform Manifold Approximation and Projection

یک nonlinear dimensionality reduction algorithm است.

ایده:

sampleهایی که در high-dimensional space همسایه‌اند، در low-dimensional representation هم تا حد امکان همسایه بمانند.

Manifold و UMAP

فرض کن یک کاغذ 2D را در 3D پیچانده‌ای:



داده در 3D است ولی intrinsic structure آن 2D است.

UMAP تلاش می‌کند manifold را کشف کند.

مراحل UMAP

ابتدا k-nearest neighbors را پیدا می‌کند.

مثلاً:

```
A → B, C, D
B → A, C, E
...
```

بعد graph می‌سازد:



و سپس یک graph مشابه در فضای 2D می‌سازد.

Optimization تلاش می‌کند این دو ساختار تا حد امکان شبیه باشند.

مهم‌ترین Hyperparameter: `n_neighbors`

مثلاً:

`n_neighbors = 15`

اگر کوچک باشد:

5

local structure مهم‌تر می‌شود.

اگر بزرگ باشد:

100

broader/global structure بیشتر لحاظ می‌شود.

`min_dist`

کنترل می‌کند points در embedding نهایی چقدر فشرده شوند.

کوچک:

••••
••••

clusterهای compact.

بزرگ:

• •
•
•

metric

Default معمولاً:

Euclidean

ولی برای embeddings:

Cosine

اغلب گزینه مهمی است.

UMAP output axes

اگر:

UMAP1
UMAP2

داشتی، اینها مثل PC1 و PC2 نیستند.

نمی‌توانی بگویی:

UMAP1 = 35% variance

چنین چیزی نداریم.

اشتباه مهم

فرض کن:

Cluster A

Cluster B

در UMAP خیلی دورند.

نباید نتیجه بگیری:

فاصله واقعی این دو group دقیقاً بسیار بزرگ است.

ارزیابی UMAP

یکی از metric های مهم:

Trustworthiness

بررسی می‌کند:

آیا همسایه‌هایی که در low-dimensional embedding دیده می‌شوند، واقعاً در original space هم همسایه بوده‌اند؟

مقدار:

$$0 \leq T \leq 1$$

بالاتر بهتر.

همچنین:

- neighborhood preservation
- stability across random seeds
- downstream clustering/classification

مهم‌اند.

مثال

BERT embeddings:

$768D$

می‌خواهی ببینی text ها structure دارند.

768D

↓

PCA → 50D

↓

UMAP → 2D

بعد plot می‌کنی.

7. t-SNE

نام کامل

t-distributed Stochastic Neighbor Embedding

یکی از مشهورترین روش‌های visualization high-dimensional data.

ایده اصلی

مثل t-SNE، UMAP هم روی neighborhood تأکید می‌کند.

اما formulation آن probabilistic است.

در high-dimensional space برای هر pair از points similarity تعریف می‌کند.

اگر دو point نزدیک باشند:

$$p_{ij}$$

زیاد است.

اگر دور باشند:

$$p_{ij}$$

کم است.

سپس در 2D چه می‌کند؟

برای low-dimensional points نیز similarity تعریف می‌کند:

$$q_{ij}$$

و تلاش می‌کند:

$$P$$

و:

$$Q$$

تا حد امکان شبیه باشند.

Loss function

از:

Kullback-Leibler Divergence

استفاده می‌کند:

$$KL(P||Q) = \sum_{i,j} p_{ij} \log \frac{p_{ij}}{q_{ij}}$$

هدف:

$$\min KL(P||Q)$$

چرا Student-t Distribution؟

اگر Gaussian برای low-dimensional similarity استفاده می‌شد، مشکل:

Crowding Problem

پیش می‌آمد.

t-SNE از heavy-tailed Student-t distribution استفاده می‌کند تا points بتوانند بهتر پخش شوند.

Perplexity

مهم‌ترین hyperparameter.

مثلاً:

perplexity=30

به شکل ساده:

تقریباً تعیین می‌کند الگوریتم چه اندازه neighborhood را مهم بداند.

Perplexity پایین:

5

بسیار local.

Perplexity بالاتر:

50

broader structure.

Rule مهم

Perplexity باید از تعداد samples خیلی کمتر باشد.

مثلاً:

N = 100 samples
perplexity = 500

معنی ندارد.

Learning Rate

Optimization t-SNE به learning rate حساس است.

خیلی کم:

points ممکن است خیلی متراکم شوند.

خیلی زیاد:

structure می تواند ناپایدار شود.

Number of Iterations

t-SNE iterative optimization دارد.

باید زمان کافی برای convergence بدهیم.

t-SNE برای چه چیزی عالی است؟

Visualization.

خصوصاً:

- image embeddings
 - gene-expression data
 - representation learned by neural networks
-

اما برای clustering مستقیم؟

احتیاط.

مثل UMAP:

clusterهای بسیار زیبا در t-SNE لزوماً cluster واقعی نیستند.

t-SNE می تواند separation visually exaggerated ایجاد کند.

یک اشتباه بسیار مهم

فرض کن:

A cluster

B cluster

نمی توانی بگویی:

A و B خیلی از هم متفاوت اند چون روی t-SNE دور هستند.

Global distances در t-SNE بسیار قابل اعتماد نیستند.

اندازه cluster هم قابل تفسیر مستقیم نیست

یک cluster بزرگ روی plot لزوماً variance واقعی بیشتری ندارد.

t-SNE در مقابل UMAP

t-SNE معمولاً:

- visualization-centric تر است
- local neighborhood بسیار خوب
- global structure ضعیف تر
- برای dataset بزرگ اغلب کندتر

UMAP اغلب:

- سریع‌تر
 - scalable
 - global structure را کمی بهتر حفظ می‌کند
 - transform کردن new samples معمولاً practical‌تر است
-

معيار t-SNE

مثل UMAP explained variance ندارد.

می‌توان:

Trustworthiness

را بررسی کرد.

همچنین neighborhood preservation و stability across seeds.

8. LDA

یک نکته حیاتی

منظور اینجا:

Linear Discriminant Analysis

است.

نه:

Latent Dirichlet Allocation

که یک topic modeling algorithm است.

LDA چه تفاوت اساسی با PCA دارد؟

PCA label ندارد.

می گوید:

directions with maximum variance.

LDA label دارد.

می گوید:

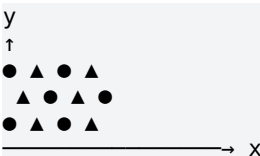
directionهایی پیدا کن که classها را تا حد امکان از هم جدا کنند.

مثال

دو class داریم:

Class A = ●
Class B = ▲

فرض کن:



ممکن است variance کلی زیاد باشد ولی separation ضعیف.

LDA directionای پیدا می کند که:

Projected:

● ● ● ● ▲ ▲ ▲ ▲

را ایجاد کند.

LDA ریاضی

دو نوع scatter داریم.

Within-Class Scatter

می‌خواهیم کوچک باشد.

$$S_W$$

یعنی اعضای هر class به هم نزدیک باشند.

Between-Class Scatter

می‌خواهیم بزرگ باشد.

$$S_B$$

یعنی class means از هم دور باشند.

LDA دنبال جهت:

$$w$$

است که تقریباً ratio زیر را maximize کند:

$$J(w) = \frac{w^T S_B w}{w^T S_W w}$$

یعنی:

زیاد	between-class	separation
		و within-class variance کم.

مثال شهودی

PCA ممکن است جهت زیر را انتخاب کند:

maximum overall variance

اما LDA:

maximum class discrimination

را انتخاب می کند.

بنابراین برای LDA، classification می تواند representation مناسب تری بسازد.

محدودیت تعداد Components در LDA

اگر:

$$C$$

class داشته باشیم، حداکثر:

$$C - 1$$

discriminant dimensions خواهیم داشت.

مثلاً اگر:

$$C = 3$$

باشد:

$$n_{components} \leq 2$$

این سؤال مصاحبه ای بسیار مهم است.

چرا؟

چون بین C class mean تنها حداکثر C-1 جهت مستقل discriminative وجود دارد.

LDA هم classifier است؟

بله.

LDA علاوه بر dimensionality reduction خودش یک classification model هم هست.

این تفاوت مهمی است.

LDA کلاسیک Assumptions

به طور کلاسیک فرض می‌شود:

هر class تقریباً Gaussian distribution دارد.

و classes covariance مشابه دارند.

یعنی:

$$\Sigma_1 \approx \Sigma_2 \approx \dots \approx \Sigma_C$$

اگر covariance خیلی متفاوت باشند، QDA یا روش دیگری شاید بهتر باشد.

LDA در Scaling

در بسیاری موارد scaling مفید است، مخصوصاً وقتی featureها scaleهای متفاوت دارند، هرچند LDA نسبت به روش‌های distance-based داستان متفاوتی دارد.

LDA چه زمانی مناسب است؟

وقتی:

- labels داری
- هدف separation classes است
- می‌خواهی قبل از classification dimensions را کاهش دهی
- supervised visualization می‌خواهی

PCA در مقابل LDA با مثال

فرض کن Cancer و Healthy داری.

PCA ممکن است variance اصلی را از Age پیدا کند:

PC1 ≈ Age

ولی Age شاید Cancer و Healthy را جدا نکند.

LDA ممکن است combination از:

Biomarker A
Biomarker B
Protein C

پیدا کند که:

Healthy Cancer
●●●● ▲▲▲▲

را بهتر جدا کند.

حالا چهار Clustering Algorithm را مستقیم مقایسه کنیم

ویژگی	K-Means	DBSCAN	GMM	Hierarchical
نوع	Centroid-based	Density-based	Probabilistic	Hierarchy-based
نیاز به K	بله	خیر	بله	لزوماً ابتدا خیر

ویژگی	K-Means	DBSCAN	GMM	Hierarchical
Noise detection	خیر	بله	مستقیم خیر	خیر
Soft membership	خیر	خیر	بله	خیر
Arbitrary shapes	ضعیف	عالی	ضعیف‌تر	بستگی به linkage
Overlapping clusters	ضعیف	متوسط	عالی	متوسط
Scale sensitivity	بالا	بسیار بالا	بالا	بالا
High-dimensional	متوسط	ضعیف	متوسط	متوسط
سرعت	معمولاً بالا	خوب	متوسط	برای n بزرگ ضعیف
معیار اصلی انتخاب	Silhouette/Elbow	DBCV/Silhouette	BIC/AIC	Dendrogram/ Silhouette
Hyperparameters کلیدی	K	eps, min_samples	K, covariance	linkage, distance

چهار Dimensionality Reduction Method

ویژگی	PCA	UMAP	t-SNE	LDA
Supervision	Unsupervised	Unsupervised	Unsupervised	Supervised
Linear	بله	خیر	خیر	بله
هدف	حفظ variance	حفظ neighborhoods/manifold	حفظ local similarity	جدا کردن classes
Visualization	خوب	عالی	عالی	عالی اگر labels داری
Global structure	نسبتاً خوب	متوسط	ضعیف‌تر	class-dependent
Local structure	متوسط	عالی	عالی	class-oriented
Explained variance	بله	خیر	خیر	به شکل PCA نه
Axes interpretable	نسبتاً	کم	کم	نسبتاً
Stochastic	تقریباً خیر	بله	بله	معمولاً خیر
Hyperparameter	n_components	n_neighbors,min_dist	perplexity	n_components/solver

ویژگی	PCA	UMAP	t-SNE	LDA
اصلی				
embeddings مناسب	preprocessing	عالی	عالی	فقط با labels
New data transform	آسان	معمولاً ممکن	سنتی محدودتر	آسان

اگر یک dataset واقعی داشته باشیم، انتخاب چطور می شود؟

فرض کن:

5000 samples

و:

768D embeddings

داریم.

اگر فقط visualization می خواهی:

768D
↓
PCA → 50D
↓
UMAP → 2D

یا:

PCA → 50D
↓
t-SNE → 2D

اگر clustering می خواهی و K تقریباً معلوم است:

Scale
↓
PCA
↓
K-Means

اگر K معلوم نیست و noise داری:

Scale
↓
PCA
↓
DBSCAN

اگر clusters overlap دارند:

Scale
↓
PCA
↓
GMM

اگر ساختار سلسله‌مراتبی برایت مهم است:

Scale
↓
Hierarchical
↓
Dendrogram

اگر labels داری و می‌خواهی class separation ایجاد کنی:

Scale
↓
LDA
↓
Classifier

یک مثال کامل پزشکی

فرض کن ۲۰۰۰ بیمار داری و برای هر بیمار:

Age
BMI
Blood Pressure
50 laboratory measurements
512-dimensional MRI embedding

داری.

در مجموع مثلاً:

565 dimensions

اول preprocessing:

Missing values
↓
Scaling
↓
Outlier inspection

بعد:

$$565D \xrightarrow{PCA} 50D$$

مثلاً 50 PC:

95%

variance را حفظ می کند.

برای visualization:

$$50D \xrightarrow{UMAP} 2D$$

و مشاهده می کنی که سه region ظاهری وجود دارد.

اما صرفاً بر اساس UMAP نمی گویی:

سه phenotype واقعی داریم.

بعد clustering روی 50D:

K-Means

K=2 تا 8 را تست می کنی.

مثلاً:

K=3 → Silhouette = 0.52
K=4 → Silhouette = 0.61
K=5 → Silhouette = 0.55

K=4 candidate خوبی است.

DBSCAN

با k-distance:

```
eps ≈ 1.2  
min_samples = 10
```

می‌گیری.

مثلاً:

```
4 clusters  
Noise = 8%  
DBCV = 0.63
```

GMM

BIC:

```
K=3 → 12000  
K=4 → 11100  
K=5 → 11350
```

پس:

$$K = 4$$

و membership probabilities به دست می‌آید.

Hierarchical

روی representative subset dendrogram می‌سازی و می‌بینی separation چهارشاخه قابل دفاع است.

اگر چند روش مختلف همگی تقریباً چهار cluster مشابه پیدا کنند، confidence تو نسبت به وجود structure واقعی بیشتر می‌شود.

مهم‌ترین نکته برای مصاحبه

هیچ الگوریتمی را نباید با جمله:

«این الگوریتم بهتر است.»

توصیف کنی.

سؤال درست این است:

بهتر برای چه نوع داده و چه هدفی؟

مثلاً DBSCAN ممکن است برای Two Moons عالی باشد و برای Gaussian overlapping populations بد. GMM دقیقاً برعکس.

PCA ممکن است برای compression عالی باشد ولی برای nonlinear visualization ضعیف‌تر از UMAP. t-SNE ممکن است plot فوق‌العاده‌ای بسازد ولی برای تفسیر global distances خطرناک باشد.

تفاوت‌هایی که باید مثل اسم خودت بلد باشی

سؤال	جواب
K-Means centroid دارد؟	بله
DBSCAN centroid دارد؟	خیر
DBSCAN K می‌خواهد؟	خیر
GMM K می‌خواهد؟	معمولاً بله
GMM membership probability می‌دهد؟	بله
DBSCAN noise پیدا می‌کند؟	بله
Hierarchical dendrogram دارد؟	بله
PCA clustering است؟	خیر
UMAP clustering است؟	خیر
t-SNE clustering است؟	خیر
PCA linear است؟	بله
UMAP nonlinear است؟	بله
t-SNE nonlinear است؟	بله
LDA supervised است؟	بله
PCA labels می‌خواهد؟	خیر
UMAP explained variance دارد؟	خیر
t-SNE explained variance دارد؟	خیر

سؤال	جواب
PCA explained variance دارد؟	بله
LDA حداکثر چند component؟	C-1
UMAP plot فاصله global دقیق می دهد؟	خیر
t-SNE cluster size را می شود literal تفسیر کرد؟	خیر

نسخه بسیار کوتاه مصاحبه‌ای هر ۸ الگوریتم

K-Means:

A centroid-based clustering algorithm that partitions observations into K clusters by minimizing within-cluster squared Euclidean distances.

DBSCAN:

A density-based clustering algorithm that identifies density-connected regions and explicitly labels sparse observations as noise without requiring the number of clusters beforehand.

GMM:

A probabilistic clustering model that assumes data arise from a mixture of Gaussian distributions and provides posterior probabilities of cluster membership.

Hierarchical Clustering:

A clustering approach that recursively merges or divides clusters to construct a multilevel hierarchy that can be visualized using a dendrogram.

PCA:

A linear unsupervised dimensionality-reduction method that projects data onto orthogonal directions of maximum variance.

UMAP:

A nonlinear manifold-learning method that builds a nearest-neighbor graph and optimizes a low-dimensional embedding designed to preserve neighborhood structure.

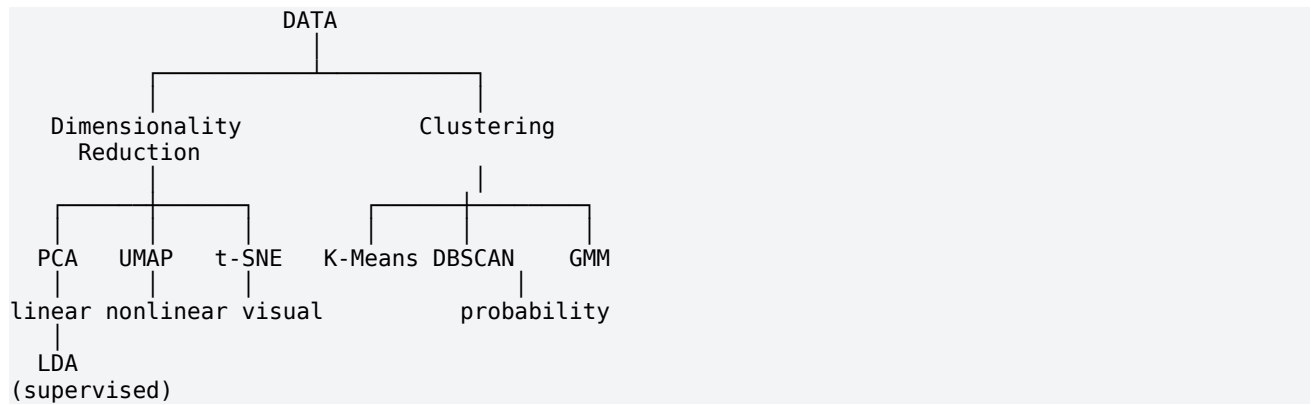
t-SNE:

A nonlinear probabilistic dimensionality-reduction method primarily used for visualization that preserves local pairwise similarities by minimizing KL divergence.

LDA:

A supervised linear dimensionality-reduction and classification method that finds directions maximizing between-class separation relative to within-class variability.

یک جمع‌بندی ذهنی خیلی خوب این است:



برای فول شدن واقعی برای مصاحبه، مرحله بعدی باید محاسبات دستی باشد. یعنی K-Means را با ۶ نقطه iteration- برای by-iteration حساب کنیم، DBSCAN را با ۱۰ نقطه و eps دستی Core/Border/Noise تعیین کنیم، GMM را یک E-step و M-step واقعی حل کنیم، Hierarchical را dendrogram دستی بسازیم، PCA را از covariance تا eigenvector عددی حل کنیم، و بعد روی همان داده UMAP/t-SNE/LDA را مفهومی تحلیل کنیم. آن بخش است که باعث می‌شود اگر مصاحبه‌کننده سؤال را از زاویه غیرحفظی بپرسد، گیر نکنی.